Claude Audit 4/29/26

Overview

Tenbin Smart Contract Audit Report

Scope: `src/**` (Solidity 0.8.30) — Controller, CollateralManager, StakedAsset, AssetSilo, AssetToken, RevenueModule, SwapModule, CustodianModule, MultiCall, GoldOracleAdapter, Gate, SpokeERC20, SpokeERC20Restricted, plus interfaces.

Tooling: Foundry only — `forge build`, `forge test` (344 / 344 passing locally), targeted reading.

Out of scope: `test/`, `script/`, `lib/`, `broadcast/`, `cache/`, CI, docs.

---

1. Architecture Summary

Tenbin issues asset-token notes (e.g., gold) backed by:

- **Off-chain** futures hedge collateralized via the `custodian` account (and `CustodianModule`).
- **On-chain** collateral held in `CollateralManager`, deposited into ERC4626 yield vaults (Morpho V2 via `Gate`).

Mint and redeem are **signed off-chain orders** processed through `Controller`:

1. KYC-approved **signer** (EOA or ERC1271 contract) signs an `Order` (EIP-712).
2. Backend **MINTER_ROLE** key (HSM) signs a `Context` containing the order hash, optional curation flag, and a share-price slippage guard.
3. Anyone can submit `(order, signature, context, approval)`. The Controller atomically:
    - Splits collateral by `ratio`: `ratio` portion → `custodian`, remainder → `manager`.
    - Optionally calls `manager.deposit` (curated mint) or `manager.withdraw` (curated redeem).

- Mints `AssetToken` to recipient or burns from payer.
- For staked-asset redeems, calls `StakedAsset.instantUnstake`, then burns the `AssetToken`.

`StakedAsset` is an upgradeable ERC4626 with vesting, cooldown (assets parked in `AssetSilo`), restricted-registry, and an instant-unstake cap. `RevenueModule` collects vault yield (PnL on share-price growth) and forwards to manager / multisig / staking rewards. `SwapModule` performs CURATOR-driven token swaps via 1inch v6.

---

2. Trust Model & Privileged Roles

Contract	Role	Powers
AssetToken	`owner` (Ownable2Step)	Sets `minter` (must be Controller in production).
AssetToken	`minter`	Unlimited mint. Drain risk if mis-set.
Controller	`DEFAULT_ADMIN_ROLE`	Add collateral, set custodian/manager, set block limits, grant other roles.
Controller	`ADMIN_ROLE`	Set ratio, oracle, rescue funds.
Controller	`SIGNER_MANAGER_ROLE`	Add/remove signers.
Controller	`MINTER_ROLE`	Off-chain HSM key signing every Context (gates every order).
Controller	`GATEKEEPER_ROLE`	Pause / unpause.
Controller	`RESTRICTER_ROLE`	Set sanctioned account list.
CollateralManager	`DEFAULT_ADMIN_ROLE`	Add/remove vaults, redeem legacy shares, change controller, upgrade implementation.
CollateralManager	`CURATOR_ROLE`	`deposit`, `withdraw`, `swap`.
CollateralManager	`REBALANCER_ROLE`	Move on-chain collateral to custodian (cap-bounded), `convertRevenue`.
CollateralManager	`CAP_ADJUSTER_ROLE`	Rebalance/swap caps and min-swap price.
StakedAsset	`DEFAULT_ADMIN_ROLE`	`transferRestrictedAssets`, upgrade.

Contract	Role	Powers
StakedAsset	`ADMIN_ROLE`	Vesting / cooldown periods, rescue tokens.
StakedAsset	`INSTANT_UNSTAKER_ROLE`	Bypass cooldown subject to cap (held by Controller).
StakedAsset	`REWARDER_ROLE`	Push rewards (resets vesting).
RevenueModule	`REVENUE_KEEPER_ROLE`	Collect, forward, set arbitrary controller approvals, send rewards.
MultiCall	`MULTICALLER_ROLE`	Arbitrary external calls from this contract.
SwapModule	`manager` (immutable)	Trigger 1inch swap.
SwapModule	`admin` (immutable)	Rescue tokens.
Gate	`owner`	Set Morpho `manager`.
CustodianModule	`DEFAULT_ADMIN_ROLE` / `CUSTODIAN_KEEPER_ROLE`	Add custodians / off-ramp.

External dependencies referenced from `src/**`

- `openzeppelin-contracts` and `openzeppelin-contracts-upgradeable` (AccessControl, Ownable2Step, ERC20, ERC20Permit, ERC4626, ECDSA, EIP712, Math, SafeERC20, ReentrancyGuardTransient, UUPSUpgradeable).
- `chainlink-local` (`AggregatorV3Interface`).
- `vault-v2` (Morpho V2 gate interfaces).
- 1inch `IAggregationRouterV6`.

---

3. Severity-ranked Findings

Severities mapped to impact × likelihood. Each finding marked as **confirmed**, **likely**, or **informational**.

H-1 (likely) — `SwapModule` partial-fill flag check is on the wrong bit

- **Impacted file:** `src/SwapModule.sol:16,82`
- **Impacted function:** `swap1Inch`

```
uint256 private constant _NO_PARTIAL_FILLS_FLAG = 1 << 255;
...
if (swapData.flags & _NO_PARTIAL_FILLS_FLAG != 0) revert Part
ialFillNotAllowed();
```

In the 1inch Aggregation Router V6 the partial-fill flag is **bit 0** (`1 << 0`), not bit 255. The constant therefore matches no real 1inch flag and the check is effectively dead code. The unit test in `test/unit/SwapModule.t.sol:178` uses `desc.flags = 1 << 255`, so it does not exercise a real partial-fill scenario.

Why it matters: The intent of the check is to refuse swap calls that allow 1inch to partial-fill. With the wrong bit checked, a curator (or anyone exploiting curator key) could submit a description with `flags & 1 != 0`. *In practice* the `if (spentAmount < params.amount) revert InvalidAmount();` post-check immediately after the swap reverts if 1inch only fills part of the order, so exploitation is currently blocked — this is the only thing keeping partial fills out. If the post-check is ever loosened or 1inch's reporting semantics change, the protection vanishes.

How it can be triggered: Any 1inch call where `swapData.flags = 1` (real partial-fill bit). Today's defense is the post-swap `spentAmount` check.

Reproduction: Modify `test/unit/SwapModule.t.sol:178` to `desc.flags = 1` and re-run `forge test --match-test test_Swap1Inch -vvv`. The current `_NO_PARTIAL_FILLS_FLAG` check will not trigger; only `spentAmount < amount` will catch a real partial fill (use the existing `Mock1InchRouterWithInsufficientAmountReportSent` to simulate).

Remediation: Replace the constant with the actual 1inch v6 flag — `uint256 private constant _PARTIAL_FILL_FLAG = 1 << 0;` — and rename to `_PARTIAL_FILL_FLAG`. Update the test to use `desc.flags = 1` so this protection is regression-tested.

M-1 (confirmed) — `StakedAsset` does not override `_decimalsOffset`

- **Impacted file:** `src/StakedAsset.sol`
- **Impacted function:** Implicit; `ERC4626Upgradeable._decimalsOffset()` returns `0`.

OZ's ERC4626 formulas are `assets.mulDiv(totalSupply + 10**offset, totalAssets + 1, ...)`. With `offset = 0` the only mitigation against share-price inflation is the `+1`

virtual offset, which is weak. The doc-string acknowledges the risk:

> In order to avoid a first depositor donation attack a minimum stake should be made in the same transaction as the contract deployment.

Even with bootstrap, two scenarios re-create the conditions for inflation (`totalSupply == 0` while the asset balance is non-zero):

1. All shares cooled down or `transferRestrictedAssets` drained while `pendingRewards > 0` leaves balance backing zero supply.
2. Future redeployments via UUPS that re-initialize state.

Why it matters: A malicious actor who lands the first deposit after a low-supply state inflates share price and steals from later depositors.

How it can be triggered: Drain (cooldown + unstake by all stakers) followed by a tiny deposit and a donation, then victim's deposit rounds to zero shares.

Reproduction: Deploy a fresh proxy without the bootstrap step, donate `1` underlying, then deposit `2` underlying — observe shares = 0 due to floor rounding of the OZ formula.

Remediation: Override `_decimalsOffset` on `StakedAsset` to return at least 6:

```
function _decimalsOffset() internal pure override returns (ui
nt8) {
    return 6;
}
```

Combine with the deployment-time bootstrap, and require a non-zero `totalSupply` invariant between upgrades.

M-2 (likely) — Asymmetric custodian/manager flow + uncapped `withdraw` during curated redeem

- **Impacted files:** `src/Controller.sol:381-389`, `src/Controller.sol:446-451`, `src/CollateralManager.sol`

`mint` only deposits `collateral_amount - custodianAmount` into the manager (the manager portion). `redeem` always calls `manager.withdraw(collateral_token,`

`order.collateral_amount, ...)` and then transfers the **full** `collateral_amount` from manager → recipient.

If the `custodian` portion has not been rebalanced back, the manager's wallet plus vault balance may be insufficient. The redeem reverts late (in `safeTransferFrom`), but with aggregate volume, the on-chain reserve drains over time. The protocol relies on `REBALANCER_ROLE` to refill via `rebalance`, which is rate-limited by a cap that is itself managed by `CAP_ADJUSTER_ROLE`. There is no on-chain solvency check tying mint/redeem flow to the real reserve ratio.

Why it matters: A burst of redemptions can cause widespread reverts and stuck users while waiting for off-chain action. A delayed or stuck rebalance can effectively "freeze" redemptions even though reserves exist off-chain.

How it can be triggered: Set `ratio = 0.7e18`, mint $1M, drain manager via several redeems until manager balance < `collateral_amount`. The next redeem reverts.

Reproduction: Multi-step Foundry integration test simulating mint + redeem sequence with mock collateral.

Remediation:

- Document the operational invariant clearly and emit a low-liquidity warning event at a configurable threshold.
- Provide a public read-only liquidity view for integrators to throttle on the front-end.
- Optionally introduce automatic auto-withdraw from the vault when manager wallet is below `collateral_amount`.

M-3 (confirmed) — Restricted accounts on `SpokeERC20Restricted` cannot have funds reclaimed

- **Impacted file:** `src/external/chainlink/SpokeERC20Restricted.sol:47-56`
- **Impacted functions:** `burn(address,uint256)`, `burnFrom(address,uint256)`

Both burn paths require `nonRestricted(account)`, so an admin cannot burn tokens of a restricted user. Unlike `StakedAsset.transferRestrictedAssets` (which allows DEFAULT_ADMIN to recover), here a restricted account's balance is permanently frozen.

Why it matters: The protocol explicitly relies on the restricted registry as a regulatory tool. The asymmetry between Ethereum-side (recoverable) and spoke-chain-side (locked) likely violates the same legal/operational expectation that drove the registry design.

How it can be triggered: Admin restricts an account that holds spoke tokens; admin attempts `burn(account, amount)`; reverts with `AccountRestricted`.

Reproduction: Restrict an account that holds tokens on `SpokeERC20Restricted`; assert that `MINTER_BURNER_ROLE` cannot burn (`AccountRestricted` revert).

Remediation: Add a DEFAULT_ADMIN-gated `forceBurn(address,uint256)` (without the `nonRestricted(account)` constraint) **or** allow `MINTER_BURNER_ROLE` to burn from restricted accounts when an explicit override flag is set, with an event.

M-4 (likely) — ERC1271 signers implicitly become payers

- **Impacted file:** `src/Controller.sol:498-513`

For ERC1271 signatures, `signer = order.payer`. The check that follows is `if (order.payer != signer && !delegates[order.payer][signer]) revert InvalidPayer();`, which trivially passes (`signer == payer`) — the only validation is that the same address is listed as a signer.

This is consistent with the comment — payers that are smart wallets must be listed as signers — but the code path also lets a smart-contract signer act as **its own** payer without being marked as a legitimate "payer-only" delegate. There is no distinction between "this contract may sign for itself" and "this contract may pay for someone else". For most deployments this is fine, but if a corporate wallet is added as a signer for operational signing, it implicitly becomes a self-payer too. Combined with that wallet's `recipients[signer][signer] = true` set automatically in `setSignerStatus`, the contract is unconditionally able to mint to itself once whitelisted.

Why it matters: Limits the granularity of access control — every signer is implicitly a fully-empowered payer for itself, even when only delegated signing was intended.

Remediation: Optionally add a per-signer flag indicating "may act as payer", and restrict ERC1271 self-payer paths accordingly. At minimum, document the implicit

equivalence.

M-5 (informational, but financially consequential) — Vesting reset on every `reward`

- **Impacted file:** `src/StakedAsset.sol:245-255`
- **Impacted function:** `reward`

```
vesting.assets = pending + assets;
vesting.end = uint128(block.timestamp) + vesting.period;
```

Each reward call extends `end` by the full `period`, so frequent small rewards keep vesting from completing. The doc-string already warns about this (rewards should be infrequent and consistent). The keeper role can also be exploited to grief stakers (delay reward distribution) or accelerate distribution (single large reward).

Why it matters: Accidental or malicious mis-scheduling of rewards can materially change effective APY for stakers.

How it can be triggered: Repeated small rewards in a tight loop reset the vesting end indefinitely.

Reproduction: Reward the staking contract many times in a tight loop; assert that `vesting.end` is always re-pushed and `_pendingRewards` never decreases below the pre-reward pending plus epsilon.

Remediation: Use a vesting model that adds new rewards to an "unvested bucket" without resetting the end time, e.g. weighted average end-time. This is a non-trivial change but worth scoping.

M-6 (informational) — `Controller.multicall` is unrestricted self-delegatecall

- **Impacted file:** `src/Controller.sol:322-331`

Anyone can call `multicall(bytes[])`. The delegated calls execute under `address(this)` storage, but with `msg.sender` preserved. Each inner call still enforces its own access control, so no privilege escalation is possible *today*. However:

- The pattern is fragile: any future externally-restricted function added to `Controller` that does not use `onlyRole` (e.g., one that depends on `tx.origin` or that interacts via internal libraries assuming a single entry-point) could be exposed.
- The Controller has multiple state-mutating public methods (`invalidateNonce`, `setRecipientStatus`, `setDelegateStatus`) that anyone can call directly; multicall does not change that, but it makes auditing of future changes more error-prone.

Remediation: Either (a) restrict `multicall` to a role (e.g., `DEFAULT_ADMIN_ROLE`) — note this changes user flow — or (b) keep public but add a comment + invariant test that every state-mutating function in Controller has explicit access control (no implicit "internal-only-via-multicall" assumptions).

L-1 (informational) — `RevenueModule.setControllerApproval` permits unbounded approval to Controller

- **Impacted file:** `src/RevenueModule.sol:129-135`

`REVENUE_KEEPER_ROLE` can call `setControllerApproval(token, type(uint256).max)`, granting Controller infinite allowance. Subsequent compromised orders with `payer = address(revenueModule)` would let Controller transfer arbitrary amounts of any approved token.

Remediation: Cap the approval at a per-call ceiling, or introduce a separate ADMIN-gated max-approval function and limit keeper to small amounts.

L-2 (informational) — `Controller` ratio applied via `mulDiv` (rounding documentation)

- **Impacted file:** `src/Controller.sol:373-383`

`custodianAmount = mulDiv(collateral_amount, ratio, RATIO_PRECISION)` rounds toward zero. The two `safeTransferFrom` calls together transfer exactly `collateral_amount`, so this is correct. No issue. Documenting only because a future refactor might compute `managerAmount = mulDiv(...)` and split asymmetrically.

L-3 (informational) — Cooldown asset-amount snapshot vs `instantUnstake` rounding

- **Impacted files:** `src/StakedAsset.sol:228-242`, `src/Controller.sol:454-459`

`instantUnstake(asset_amount, payer, payer)` computes `shares = previewWithdraw(asset_amount)` (rounds up) and then forwards to `super.redeem(shares, payer, payer)` which transfers `previewRedeem(shares)` (rounds down). That value can be `asset_amount - 1 wei`. The Controller then does `AssetToken.burn(payer, asset_amount)`. If the payer has zero idle `AssetToken` balance, the burn reverts.

Why it matters: Edge-case UX issue. The user may need a wei of asset balance for a clean redeem. Not exploitable.

Remediation: Either (a) burn the actual unstaked amount returned by `instantUnstake` (change `instantUnstake` to return the realized assets and use that), or (b) document the requirement.

L-4 (informational) — `removeCollateral` zeroes accounting state but vault shares survive in manager

- **Impacted file:** `src/CollateralManager.sol:176-186`

After `removeCollateral`, the manager may still hold shares of the removed vault. `redeemLegacyShares` reclaims the underlying; the underlying is then rescuable via `rescueToken`. Operationally fine, but worth tracking in run-books — without `redeemLegacyShares + rescueToken` the funds are stranded.

L-5 (informational) — `setIsCollateral` (Controller) and `addCollateral` (Manager) are independent

- **Impacted files:** `src/Controller.sol:243-252`, `src/CollateralManager.sol:155-170`

If admin removes a token from Controller's whitelist before removing from Manager (or vice versa), in-flight orders may revert deep in the call stack. Recommend an admin script that performs both atomically.

L-6 (informational) — `AssetSilo.asset.approve(staking, max)` uses raw `approve`

- **Impacted file:** `src/AssetSilo.sol:29`

Safe for AssetToken (standard ERC20Permit) but a minor hygiene issue if the silo is ever instantiated for a non-standard asset. Convert to `SafeERC20.forceApprove`.

L-7 (informational) — `setBlockMintLimit` / `setBlockRedeemLimit` emit no events

- **Impacted file:** `src/Controller.sol:294-301`

Off-chain monitoring is harder. Add events.

L-8 (informational) — Loop counters typed `uint16`

- **Impacted file:** `src/CollateralManager.sol:209,292`

Theoretical revert at 65,535 collaterals; impractical, but pin to `uint256` for consistency.

L-9 (informational) — `signers[address(0)]` can be set true

- **Impacted file:** `src/Controller.sol:201-205`

OZ's `ECDSA.recover` reverts on malformed signatures rather than returning `address(0)`, so this is not exploitable. Add `nonZeroAddress(account)` for hygiene.

L-10 (informational) — Duplicate `burn` and `burnFrom` definitions

- **Impacted files:** `src/AssetToken.sol:62-71`, `src/external/chainlink/SpokeERC20.sol:34-43`, `src/external/chainlink/SpokeERC20Restricted.sol:47-56`

`burn(address,uint256)` and `burnFrom(address,uint256)` are functionally identical. Pick one (CCIP `IBurnMintERC20` mandates both, so this is intentional for cross-chain compatibility — confirm with the CCIP integration).

L-11 (informational) — `convertRevenue` and `rebalance` lack `nonReentrant`

- **Impacted file:** `src/CollateralManager.sol:362-378`

Neither makes a callback-prone external call (`safeTransfer` on standard ERC20 tokens is safe), so no exploit. Adding `nonReentrant` is defense-in-depth and consistent with the rest of the contract.

L-12 (informational) — `GoldOracleAdapter.getPrice` only checks `updatedAt`

- **Impacted file:** `src/oracle/GoldOracleAdapter.sol:30-40`

Modern Chainlink aggregators fold staleness into `updatedAt`, but historically a reverted round could return non-stale data. Add `if (answeredInRound < roundId) revert OraclePriceStale();` if regulatory cover is desired.

L-13 (informational) — `MultiCall.multicall` has unbounded `.call` to arbitrary targets

- **Impacted file:** `src/MultiCall.sol:22-32`

`MULTICALLER_ROLE` is the trust boundary; the role can call any contract, with all approvals granted to MultiCall. The role must therefore be held by a maximally trusted curator. Document this explicitly. If the role is ever broadened, treat as critical.

L-14 (informational) — `Gate.setManager` allows the zero address

- **Impacted file:** `src/external/morpho/Gate.sol:20-22`

Setting to zero blocks all vault interaction. May be intentional as a kill switch, but add a `nonZeroAddress` (or a dedicated `pause` flag) for clarity and emit an event.

---

4. Open Questions / Assumptions

1. **MINTER_ROLE custody.** The audit assumes `MINTER_ROLE` is held only by an HSM-protected backend key and is not mass-granted. Compromise of this key, combined with any KYC'd signer, allows arbitrary mint and redeem orders.
2. **Off-chain solvency.** The protocol relies on operators (REBALANCER_ROLE) to maintain on-chain liquidity. There is no on-chain enforcement that the off-

chain hedge remains solvent or that funding cost ≤ vault yield. A loss off-chain is not visible on-chain; the on-chain accounting only tracks vault PnL.

3. **Vault trust.** ERC4626 vaults registered via `addCollateral` receive infinite approval. The audit treats vault implementations (Morpho V2 + Gate) as trusted infrastructure.
4. **1inch trust.** The 1inch v6 router is treated as trusted. If 1inch changes their flag layout in a router upgrade, see H-1.
5. **Restricted-registry policy.** The audit assumes restricting an account is a regulatory action. The asymmetry between StakedAsset (recoverable) and SpokeERC20Restricted (not recoverable) should be deliberate — see M-3.
6. **Curator approval to vault is `type(uint256).max`.** Acceptable given vault trust assumption (item 3) but is a high blast-radius approval; consider rotating with each `setSwapModule` / vault rotation.
7. **`ratio = 0` and `ratio = RATIO_PRECISION` edges.** Zero sends everything on-chain; one sends everything off-chain (manager receives nothing). At `ratio = 1e18` curated mint is skipped (`currentRatio < RATIO_PRECISION` is false), and curated redeem still attempts a full vault withdrawal. Confirm with operations team that `ratio` near `1e18` is operational.
8. **`nonces[payer][nonce] = true` is set before some checks.** All checks that follow either accept the order or revert (rolling back the storage write), so replay protection is sound.
9. **Vesting reset behaviour (M-5)** is intentional per docs; confirm that operations enforces the once-per-day rewarding cadence in production.

---

5. Residual Risks

Risk	Mitigation in code	Residual
Compromised signer + minter key	EIP-712 + KYC + HSM	Full drain possible
Compromised CURATOR_ROLE	Swap caps, min-swap-price, pause	Bounded loss per cap window

Risk	Mitigation in code	Residual
Compromised REBALANCER_ROLE	Per-collateral cap	Bounded loss = sum of cap top-ups
Compromised DEFAULT_ADMIN_ROLE	Multisig assumption	Full re-config / upgrade — total loss
Compromised MULTICALLER_ROLE	Granted only to curators	Arbitrary calls from MultiCall context
Off-chain hedge insolvency	Off-chain	Not detectable on-chain
Vault loss > pendingRevenue	`_getRevenue` floors at zero	Unrealized loss is silently absorbed into baseline
1inch upgrade changes flag layout	Post-swap `spentAmount` check	Partial fill may slip past flag check (H-1)
ERC4626 inflation attack	Bootstrap stake, +1 offset	Weak; M-1

6. Foundry-based Reproduction Pointers

For each material finding, the recommended reproduction approach is:

- **H-1 (partial-fill flag):** Modify `test/unit/SwapModule.t.sol:178` to set `desc.flags = 1` (1inch real partial-fill bit) and run `forge test --match-test test_Swap1Inch -vvv`. The current `_NO_PARTIAL_FILLS_FLAG` check will not trigger; only `spentAmount < amount` will catch a real partial fill (use the existing `Mock1InchRouterWithInsufficientAmountReportSent` to simulate).
- **M-1 (decimals offset):** Write a test that deploys a fresh `StakedAsset` proxy without bootstrap, calls `IERC20(asset).transfer(staking, 1)` (donation), then a victim deposits a small amount and observes `balanceOf(victim) == 0`.
- **M-2 (liquidity exhaustion):** Write an integration test that mints with `ratio = 0.7e18`, then submits redeems until the manager wallet balance is below the next `collateral_amount`; verify the next redeem reverts in `safeTransferFrom`.
- **M-3 (frozen restricted balance on spoke):** Restrict an account that holds tokens on `SpokeERC20Restricted`; assert that `MINTER_BURNER_ROLE` cannot burn (`AccountRestricted` revert).

- **M-5 (vesting reset DoS):** Reward the staking contract many times in a tight loop; assert that `vesting.end` is always re-pushed and `_pendingRewards` never decreases below the pre-reward pending plus epsilon.

---

7. Recommended Fix Priority

1. **H-1** Fix partial-fill flag to bit 0; update tests (small, mechanical change).
2. **M-3** Add admin-gated burn for restricted accounts on spoke chains.
3. **M-1** Override `_decimalsOffset` in `StakedAsset` (requires upgrade-storage-layout review).
4. **M-2** Add liquidity-status events / view, or auto-withdraw fallback when wallet < required.
5. **L-1** Replace unbounded `setControllerApproval` with capped variant.
6. **L-7, L-9, L-12, L-14** Hygiene: events on missing setters, `nonZeroAddress`, oracle round validation, gate zero-check.
7. **M-5, M-6, L-13** Re-evaluate role assignments and document trust boundaries.

No critical findings were identified in this pass. The protocol's primary risk surface is operational (key custody, off-chain hedge solvency) rather than direct contract-level vulnerability.

Audit conducted strictly within `src/**` using Foundry. Comments are treated as guidance only; no test was used as a proof of correctness. Build clean (`forge build`), test clean (344 passed). No issues prevented compilation or test execution.